PRACTICAL LAB: JAVASCRIPT OPERATORS MASTERY

Course: DFK20013 Web Design Technology

Topic: 3.1.1 Variables, Data Types, and Operators

Duration: 60 – 90 Minutes

Student Name	**MUHAMMAD NAUFAL DANISH BIN MOHAMAD FAIZAL**
Registration No	**10DIT25F1043**
Date	**2/3/2026**

---

1. Learning Outcomes

By the end of this lab, students should be able to:

- Implement **Arithmetic Operators** for calculations.
- Use **Assignment Operators** to update values efficiently.
- Control application states using **Increment/Decrement**.
- Validate user access using **Relational** and **Logical** operators.

---

2. Practical Task: The All-in-One Operator Workshop

Create a file named lab_operators.html. Copy and complete the coding below.

HTML

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>DFK20013: JS Operators Workshop</title>

    <style>

        body { font-family: 'Segoe UI', sans-serif; max-width: 850px;
margin: 2rem auto; background-color: #f0f2f5; padding: 20px; color: #333;
}

        section { background: white; padding: 25px; border-radius: 12px;
margin-bottom: 25px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }

        h2 { color: #2c3e50; border-left: 5px solid #3498db; padding-
left: 10px; margin-top: 0; }

        .output-box { background: #2c3e50; color: #27ae60; padding: 15px;
border-radius: 6px; font-family: 'Courier New', monospace; margin-top:
10px; }
```

```
        .short-note { background: #fff3cd; color: #856404; padding: 15px;
border-radius: 6px; margin-top: 15px; border-left: 5px solid #ffc107;
display: none; font-size: 0.9rem; }

        input { padding: 8px; width: 100px; border-radius: 4px; border:
1px solid #ccc; }

        button { padding: 8px 20px; background: #3498db; color: white;
border: none; border-radius: 4px; cursor: pointer; font-weight: bold; }

        button:hover { background: #2980b9; }

        code { background: #eee; padding: 2px 4px; border-radius: 3px;
font-weight: bold; color: #d63384; }

    </style>

</head>
```

```
<body>



    <header style="text-align: center; margin-bottom: 30px;">

        <h1>JavaScript Operators Mastery</h1>

        <p>Follow the steps and observe the Console and UI outputs.</p>

    </header>
```

```
    <section>

        <h2>Task 1: Basic Arithmetic (Check Console)</h2>

        <p>This task demonstrates background calculations using
<code>+</code>, <code>-</code>, <code>*</code>, <code>/</code>,
<code>**</code>, and <code>%</code>.</p>

        <button onclick="runTask1()">Run Console Logic</button>

    </section>
```

```
    <section>

        <h2>Task 2: POS System (Arithmetic & Assignment)</h2>

        <p>Price: RM 50.00. <strong>Promo:</strong> Buy > 4 units to get
RM 20.00 discount.</p>

        Quantity: <input type="number" id="qty" value="1">

        <button onclick="runTask2()">Calculate Total</button>

        <div id="outPOS" class="output-box">Final Total: RM 0.00</div>

        <div id="notePOS" class="short-note"></div>

    </section>
```

```
    <section>

        <h2>Task 3: Stock Control (Increment & Decrement)</h2>
```

```
        <p>Simulating inventory updates.</p>

        Stock Level: <span id="stockVal" style="font-size: 1.5rem; font-
weight: bold;">10</span><br><br>

        <button onclick="runTask3('inc')">Add (+)</button>

        <button onclick="runTask3('dec')"
style="background:#6c757d;">Remove (-)</button>

        <div id="noteStock" class="short-note"></div>

    </section>
```

```
    <section>

        <h2>Task 4: Gate Access (Relational & Logical)</h2>

        <p>Rules: Age 18+ OR (Age 13+ AND Parent present).</p>

        Age: <input type="number" id="age" placeholder="Age">

        <label><input type="checkbox" id="parent"> With
Parent?</label><br><br>

        <button onclick="runTask4()" style="background:#27ae60;">Verify
Entry</button>

        <div id="outSecurity" class="output-box">Status: Waiting...</div>

        <div id="noteSecurity" class="short-note"></div>

    </section>
```

Inside the <script> tag, type all the following code:

JAVASCRIPT:

1.

```
<script>
    // Helper to show explanations
    function showNote(id, text) {
        const noteDiv = document.getElementById(id);
        noteDiv.style.display = 'block';
        noteDiv.innerHTML = text;
    }
```

2.

```
// --- TASK 1: ARITHMETIC (BASIC) ---
function runTask1() {
    console.log("--- Task 1: Arithmetic Operations ---");
    let students = 31;
    students = students + 1;   // Addition
    students = students - 2;   // Subtraction
    students = students * 2;   // Multiplication
    students = students / 2;   // Division
    let power = 10 ** 2;       // Exponentiation (100)
    let remainder = 31 % 2;    // Modulus (1 - Odd)

    console.log("Final Student Count: " + students);
    console.log("10 to the power of 2: " + power);
    console.log("Remainder of 31/2: " + remainder);
    alert("Check Browser Console (F12) for results!");
}
```

3.

```
// --- TASK 2: POS SYSTEM (ARITHMETIC & ASSIGNMENT) ---
function runTask2() {
    let price = 50;
    let qty = parseInt(document.getElementById('qty').value);

    // a. Arithmetic Operator (*)
    let total = price * qty;
    let note = "<b>Subtopic (a) Arithmetic:</b> <code>*</
    code> multiplied " + price + " by " + qty + ".<br>";

    // b. Assignment Operator (-=)
    if (qty > 4) {
        total -= 20; // Shorthand for total = total - 20
        note += "<b>Subtopic (b) Assignment:</b> <code>-=</
        code> updated the total by subtracting the discount.";
    }

    document.getElementById('outPOS').textContent = "Final
    Total: RM " + total.toFixed(2);
    showNote('notePOS', note);
}
```

4.

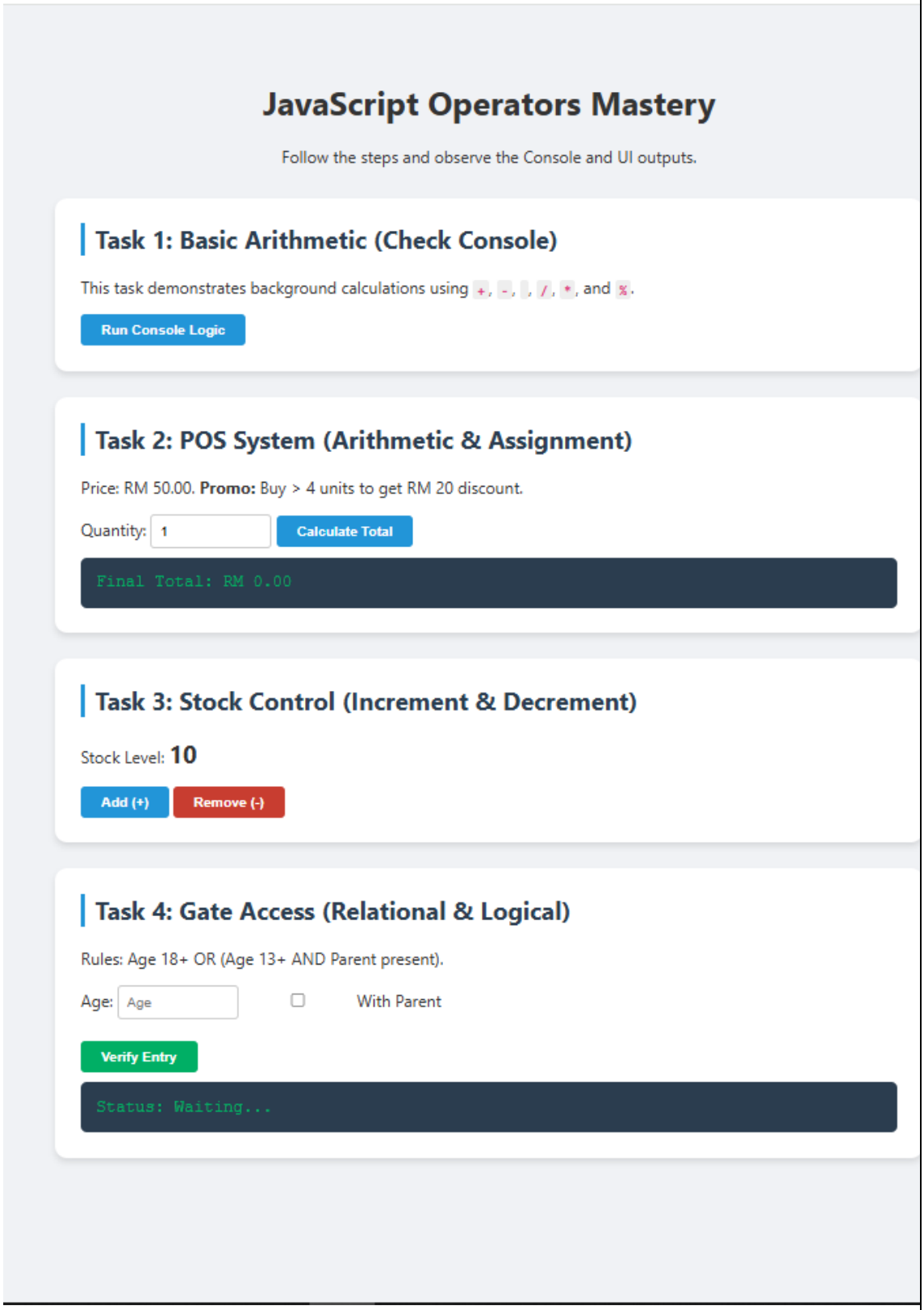
```
// --- TASK 3: INCREMENT & DECREMENT ---
let stock = 10;
function runTask3(type) {
    let note = "";
    if (type === 'inc') {
        stock++; // (c) Increment
        note = "<b>Subtopic (c) Increment:</b> <code>++</
        code> increased the stock by 1.";
    } else {
        stock--; // (c) Decrement
        note = "<b>Subtopic (c) Decrement:</b> <code>--</
        code> decreased the stock by 1.";
    }
    document.getElementById('stockVal').textContent = stock;
    showNote('noteStock', note);
}
```

5.

```
    // --- TASK 4: RELATIONAL & LOGICAL ---
    function runTask4() {
        let age = parseInt(document.getElementById('age').value);
        let hasParent = document.getElementById('parent').checked;
        let status = document.getElementById('outSecurity');

        if (isNaN(age)) return alert("Please enter a valid age.");

        // d. Relational: >=
        // e. Logical: || (OR), && (AND)
        let isEligible = (age >= 18) || (age >= 13 && hasParent);

        status.textContent = isEligible ? "Status: ACCESS GRANTED
        ✅" : "Status: ACCESS DENIED ❌";
        status.style.color = isEligible ? "#27ae60" : "#ff4d4d";

        let note = "<b>Subtopic (d) Relational:</b> <code>>=</
        code> compared Age (" + age + ") to the limit.<br>";
        note += "<b>Subtopic (e) Logical:</b> Used <code>||</
        code> (OR) and <code>&&</code> (AND) to combine rules.";
        showNote('noteSecurity', note);
    }
</script>
```

Capture a screenshot of the output from the previous steps. **Observe** the results and **explain** your understanding of the process.

1.	**Screenshot of the output**



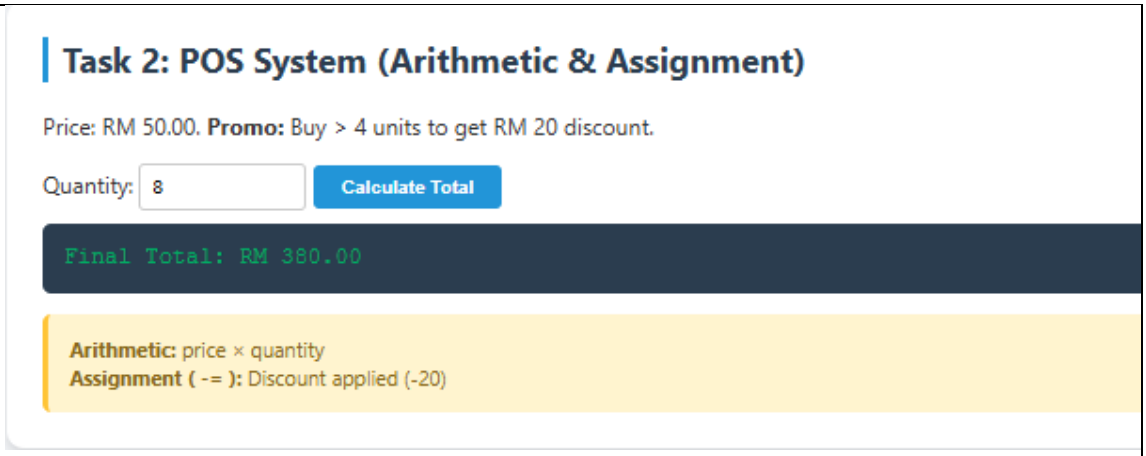
	Explanation : This interface demonstrates how different JavaScript operators work through practical mini tasks. Each task focuses on a specific category of operators, allowing users to observe both console outputs and user interface (UI) changes. The system integrates arithmetic, assignment, increment/decrement, relational, and logical operators in a structured and interactive manner. The design helps reinforce conceptual understanding by linking theory to visible output results.

2.	**Screenshot of the output :**

```
<!DOCTYPE html>
<html lang="en">
 ▸ <head> ⋯ </head>
··· ▾ <body> == $0
    ▸ <header style="text-align:center; margin-bottom:30px;"> ⋯ </header>
      <!-- TASK 1 -->
    ▸ <section> ⋯ </section>
      <!-- TASK 2 -->
    ▸ <section> ⋯ </section>
      <!-- TASK 3 -->
    ▸ <section> ⋯ </section>
      <!-- TASK 4 -->
    ▸ <section> ⋯ </section>
    ▸ <script> ⋯ </script>
  </body>
</html>
```

	Explanation : This screenshot shows the HTML structure of the webpage. The document begins with the `<!DOCTYPE html>` declaration, followed by the `<html>`, `<head>`, and `<body>` elements. Inside the `<body>`, the page is divided into sections for each task: • **Task 1** • **Task 2** • **Task 3** • **Task 4** Each task is organized within a `<section>` tag for clarity and separation of content. At the bottom, the `<script>` tag contains the JavaScript logic that controls calculations, button actions, and condition checking.

3.	**Screenshot of the output**



Explanation : This task simulates a simple Point of Sale (POS) system.

- The price per unit is RM50.00.
- A promotion applies: if the quantity is more than 4 units, a RM20 discount is given.
- In the screenshot, the quantity entered is 8.

Calculation:

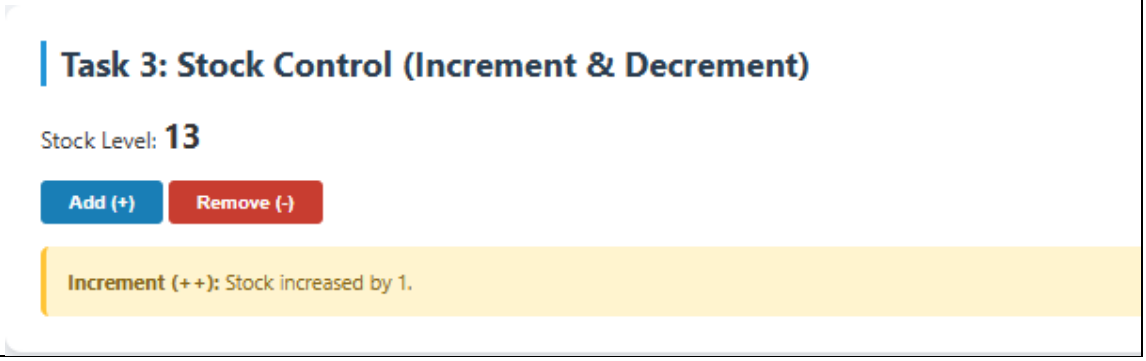
- Basic arithmetic operation:
  `Total = price × quantity`
  `Total = 50 × 8 = 400`
- Since 8 > 4, a discount of RM20 is applied using the assignment operator (-=).
- Final Total = 400 − 20 = RM380.00

This task demonstrates:

- Arithmetic operator (*)
- Assignment operator (-=)

It shows how business logic can be implemented using simple mathematical and conditional operations.

4. **Screenshot of the output**



Explanation : This task manages stock levels using increment and decrement operators.

- Initial stock level changes to 13 after pressing the "Add (+)" button.
- The increment operator (++) increases the stock value by 1 each time it is triggered.
- The decrement operator (--) would reduce the stock by 1 when "Remove (-)" is pressed.

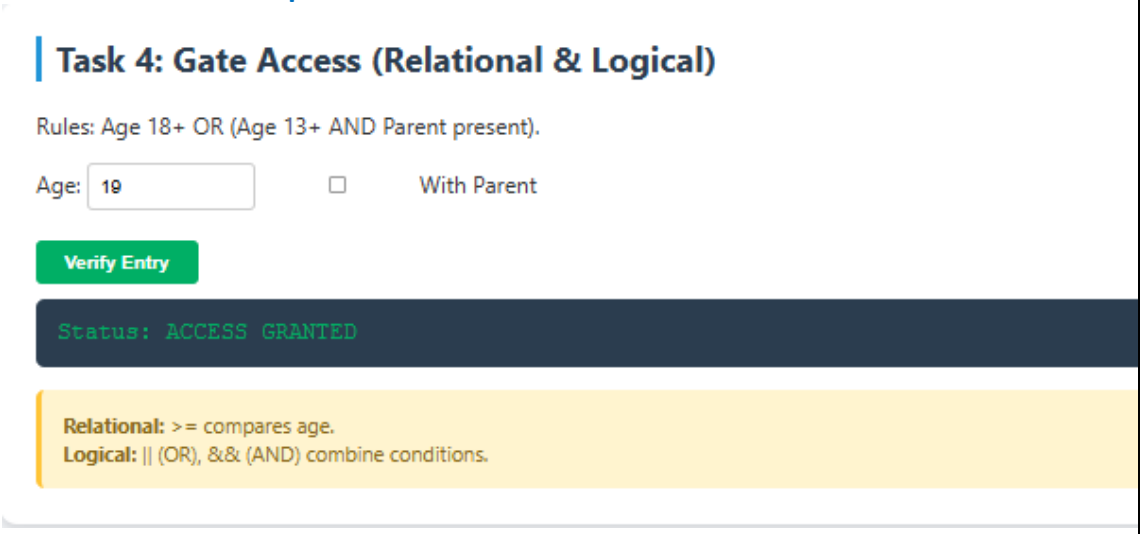
This demonstrates:

- Increment operator (++)
- Decrement operator (--)

These operators are commonly used in inventory systems, counters, and loops.

5.

Screenshot of the output



Explanation : This task verifies entry based on age conditions.

Rule:

- Age ≥ 18
  OR
- Age ≥ 13 AND parent present

In the screenshot:

- Age entered: 19
- Since 19 ≥ 18, access is automatically granted.
- The system displays: **"ACCESS GRANTED"**

Operators used:

	• **Relational operator:** `>=` (compares numerical values)
	• **Logical operators:**
	  ○ `||` (OR)
	  ○ `&&` (AND)
	
	This demonstrates how multiple conditions can be combined to produce a final decision. It reflects real-world conditional validation systems such as ticketing, age-restricted access, or authentication rules.

3. Quick Check (Reflection)

Instruction: Answer the following questions based on your coding results.

1. **Modulus Utility:** Based on Task 1, if students % 2 returns 0, is the number even or odd?

   If students % 2 returns 0, the number is even.

   Explanation:

   The modulus operator (%) gives the remainder after division.

   If a number divided by 2 has no remainder (remainder = 0), it means the number is evenly divisible by 2 — therefore, it is an even number.

2. **Shorthand Power:** In Task 2, rewrite total -= 20 using the standard assignment method.

   avaScript

   total -= 20;

   Using standard assignment:

   JavaScript

   total = total - 20;

   Explanation:

   The -= operator is a shorthand assignment operator. It subtracts 20 from total and stores the result back into total.

3. **Logical Priority:** In Task 4, why did we put (age >= 13 && hasParent) in parentheses?

   We put (age >= 13 && hasParent) in parentheses to control logical grouping.

   Explanation:

   The full condition is:

   JavaScript

   age >= 18 || (age >= 13 && hasParent)

   The parentheses ensure that:

   age >= 13 AND hasParent are evaluated together first.

   Then the result is compared with age >= 18 using OR (||).

   Without parentheses, the logic could be misunderstood or evaluated incorrectly. Parentheses make the intended rule clear and readable.

4. **Time Calculation:** (Extra Challenge) If a variable totalMinutes = 130, write the formula to find the remaining minutes after converting to hours using the **Modulus** operator.

   If:

   JavaScript

totalMinutes = 130;

To find the remaining minutes after converting to hours:

JavaScript

remainingMinutes = totalMinutes % 60;

Explanation:

There are 60 minutes in 1 hour.

130 ÷ 60 = 2 hours remainder 10 minutes.

The modulus operator (%) returns the remainder, which is 10 minutes.